
Website — Guest & Host Findings

A finding-by-finding response to the Final Website Findings before final testing: what the platform does today, measured against the live site and the live API, and the nine requirements that were not met when we checked.

Aajoo Homes — Website · Guest & Host Findings

Prepared 6 September 2026 by the ZypheX Tech development team

Verified against the live development backend and database

1. Response summary

This answers **AAJOO HOMES — Final Website Findings, Guest & Host Required Fixes Before Final Testing**, section by section and ID by ID, in the order the document raises them. Every ID is carried over unchanged.

The document is a checklist of about 110 requirements rather than a list of observed failures, and we have treated it that way: each row below says what the platform does today and **how we know** — measured against the live site and the live API where that was possible, and read from the code where it was not, which is stated in every case.

Nine of its requirements were not met when we checked — eight defects and one missing feature. One of the eight was an unauthenticated endpoint on production that would send an email from the Aajoo Homes account to any address given to it. All nine were dealt with during this review and are described in section 19.

Response at a glance	
Findings answered	6 P0 + ~110 numbered IDs across 19 sections
Already implemented, verified	~100
Fixed during this review	8
Built during this review	1 — guest support tickets (S-03, S-04, S-05, API-04)
Needs your input before we can build it	1 — the API error-code table (API-07)
Backend	1effc41
Frontend	b8f3818
Android	1.0.0 (29)
Migration applied to the live database	20260906120000-guest-support-tickets
Backend test suite	60/60 files pass

On evidence. Anything marked *verified live* was measured on `www.aajoohomes.com` or `aajaodev.onrender.com` on 6 September 2026 — response codes, response headers, rate-limit headers, rendered widths and API payloads are quoted as they came back. Authenticated Guest and Host flows were verified **from the code**, because we do not type stored account passwords into forms; where a row could only be checked that way, it says so.

2. Section 2 — Release-blocking P0

ID	Finding	Status	What is true now
WEB-P0-01	<code>/property/detail/undefined</code> is reachable	Fixed — both halves	The URL already answered a real 404 (verified live), but the site was still <i>generating</i> it: the featured grid maps <code>id</code> from <code>property_id</code> , and a row without one produced a link to a property that cannot exist. Every place that interpolates an id into a property URL now checks it first. One placeholder grid was linking to <code>\${i + 1}</code> — a card's <i>position</i> published as a property id — and now links nowhere. <code>/home</code> , the retired pre-redesign page that carried that grid, redirects to <code>/</code> . Sitemaps have never contained such URLs: they are built from approved listings only.
WEB-P0-02	Negotiation must be a complete server-connected flow	Done	Offer → server decision → host response → agreed price → booking → payment is a server flow end to end. The agreed price reaches checkout as a coupon the server issues and pins; the client cannot name it. Auto-accept at or above the floor, escalation below it, expiry, attempt limits and a full offer ledger all live server-side. See section 7.
WEB-P0-03	Monthly stay flow complete wherever monthly is offered	Done	Weekly and monthly rates are part of the pricing engine and are applied by the server for any stay long enough to qualify, on both normal and advance bookings. Renewal is month-by-month through a new booking rather than a separate renewal object — the same path a first month takes, so there is no second code path to keep correct.
WEB-P0-04	Booking, payment and confirmation must be one authoritative transaction	Done	Availability → booking → payment → server-side signature verification → confirmation, and the verification handler takes a row lock on the payment before it does anything. A replayed callback — a gateway retry, a double-tap, a deliberate replay — carries a valid signature, so the lock is the idempotency point: the second arrival waits, then sees "already paid" and returns success with no side effects . Booking creation serialises per property, so two guests cannot both win the same nights.
WEB-P0-05	Cancellation and refund flow complete	Done	My Bookings → Cancel → the applicable policy shown before confirming → email OTP → server-calculated refund → booking state updated → refund status visible. The refund is computed from the policy snapshot taken at booking, so a later policy change cannot alter a guest's entitlement, and check-in is measured in IST rather than in the server's timezone.
WEB-P0-06	Guest and Host authorization enforced server-side	Done, and tightened	Every guest query is scoped by the id on the token; every host query by <code>property_host_id</code> . A booking id, property id or ticket id in a request is a lookup key and never an authorisation. Two things were tightened during this review: the public property endpoint stopped returning the host's own phone number and email, and payout requests are now checked against the balance. See sections 17 and 16.

3. Section 3 — Guest authentication and account

ID	Requirement	Status	Evidence
G-01	OTP send → verify → session	Done	Code + live: the OTP endpoints answer and are rate-limited.

ID	Requirement	Status	Evidence
G-02	OTP abuse controls	Done — tightened this review	Three separate controls: 5 attempts per code (<code>otpMaxAttempts</code>), 10-minute validity, and a request budget that was running at 50 per 15 minutes and is now 5 , matching the attempt limit so neither quietly outlives the other. Verified live: the endpoint answers <code>ratelimit-limit: 5</code> .
G-03	Expired session must not leave a false authenticated state	Done	An expired token is rejected server-side and the client clears its session on a 401 rather than continuing to render a signed-in shell.
G-04	Logout revokes the server session	Done	Logout used to write a log row and return "Logout successful" while the 30-day JWT kept working — deleting the client's copy <i>was</i> the logout. There is now a real revocation store, covered by <code>tests/sessionRevocation.test.js</code> .
G-05	Protected routes	Done	<code>RenterRoute</code> , <code>HostRoute</code> and <code>AdminProtectedRoute</code> gate the client, and every corresponding endpoint gates the server. The client guard is convenience; the server guard is the control.
G-06	Account deletion	Done	<code>POST /user/delete</code> , authenticated. Bookings and financial records are retained rather than hard-deleted, because a deleted booking is a deleted invoice.
G-07	Profile loads from the backend and persists	Done	And revalidates: a signed-in tab now refreshes the profile in the background, so an admin's change (a KYC approval, a name) reaches an open session instead of waiting for a sign-out.
G-08	Privacy/Terms consent recorded; both accessible	Done	Per-category cookie consent since 5 September — the earlier banner asked about analytics and then loaded a pixel and Hotjar regardless. Vendor scripts now load only for the categories accepted. Privacy Policy and Terms are public pages.

4. Section 4 — Guest search and property discovery

Seven silent search defects were found and fixed on 5 September, before this document arrived; the rows below reflect the state after that work.

ID	Requirement	Status	Evidence
G-09	Results correspond to the selected location	Done	Geo-bounded server-side.
G-10	Availability uses the selected dates	Done	Booked and host-blocked ranges are excluded by the server, not filtered on the page.
G-11	Guest count validated against capacity	Done	Server-side, against the property's own capacity. The "Who" segment genuinely filters.
G-12	Filters affect returned results, not just UI state	Done — this was one of the seven	Type and count filters were narrowing <i>the page already fetched</i> rather than the catalogue, so a filter appeared to work and quietly hid matching stays. Both are server-side now.

ID	Requirement	Status	Evidence
G-13	Unavailable properties/dates not bookable	Done	Enforced at booking creation as well as in search — the page cannot be the control.
G-14	Inactive/deleted/unpublished not bookable or exposed	Done	<code>is_active = 1 AND is_deleted ≠ 1</code> is the single visibility rule, applied by search, by the property endpoint, by the sitemap and by the public page — which answers 404 , verified live on a draft listing.
G-15	Details from authoritative data	Done	Price, amenities, rules, location and availability all come from the listing's own tables.
G-16	Images load and belong to the property	Done	No stock fallback: an earlier build filled empty listings with stock photographs, which advertised places that do not exist. A listing with no photographs now looks like one.
G-17	Reviews belong to the property and follow the rules	Done	Scoped by property; only approved, active reviews are shown, and only real ones feed the rating.
G-18	Malformed/undefined/nonexistent ids resolve cleanly	Done	Verified live: <code>/property/detail/undefined</code> , <code>/property/undefined</code> and an unknown slug all answer 404 with <code>noindex</code> .

5. Section 5 — Guest booking and availability

ID	Requirement	Status	Evidence
G-19	No double booking of the same inventory	Done	Bookings serialise per property before the overlap guards run. Without it two requests a millisecond apart both read "no overlap" and both insert.
G-20	Displayed, negotiated, tax and payable amounts consistent with the server	Done	The server recomputes and rejects a client figure that differs by more than one rupee — verified live: a booking posted with a stale price is refused with "The price for these dates has changed."
G-21	Guest A cannot reach Guest B's booking	Done	Every booking query carries <code>book_user_id</code> from the token.
G-22	Booking detail correct	Done	Property, dates, guests, amount, status and payment state, from the booking row.
G-23	Upcoming/ongoing/past/cancelled accurate	Done	The status set was corrected in an earlier pass — a bug in it had hidden a guest who was in residence.
G-24	Modification respects status and policy	Done	Refused once the stay has started, and once cancelled.
G-25	Invoice available and matching	Done	<code>GET /user/invoice/:bookId/download</code> , authenticated and owner-scoped. Invoice files are no longer served as public static assets — they were, and an invoice names the guest, the property, the dates and the amount.

ID	Requirement	Status	Evidence
G-26	Payment status server-authoritative	Done	Written only by the verification handler and by the gateway webhook.

6. Section 6 — Guest negotiation

ID	Requirement	Status	Evidence
N-01	Offer creation for an eligible property	Done	Eligibility is a server decision — a discounted listing, an advance booking and a listing with negotiation off are all refused with a reason.
N-02	Guest never receives minimum/ideal thresholds	Done	<code>property_mini_price</code> and <code>property_ideal_price</code> are excluded for anyone but the owner. Verified live on the public property payload: both absent.
N-03	At or above minimum follows the auto-accept rule	Done	Server-side, from the listing's own floor.
N-04	Below minimum escalates to the host	Done	With the host's allowed responses and a deadline.
N-05	Host responds only for an authorised property	Done	Scoped by <code>property_host_id</code> .
N-06	Accepted price becomes the authoritative booking price	Done	Issued as a server-side coupon and pinned to the offer's dates — the client cannot name a price or move the dates the deal was struck for.
N-07	Expired offer cannot be accepted or reused	Done	Expiry is checked server-side on every action.
N-08	Duplicate offers cannot create inconsistent records	Done	The guard and the round number come from one locking read held until the offer row is written; without it two requests a millisecond apart both saw no pending offer.
N-09	Concurrent negotiation is deterministic	Done	Same lock.
N-10	Reconnect must not duplicate or lose the final state	Done	The final state lives in the offer row, not in the socket; a reconnect re-reads it.
N-11	Offers, decisions and outcomes recorded	Done	A full ledger, including abandoned and expired outcomes.

7. Section 7 — Guest payment

ID	Requirement	Status	Evidence
P-01	Amount tampering in the browser	Rejected	The server recomputes; a mismatch beyond ₹1 is refused.

ID	Requirement	Status	Evidence
P-02	Payment belongs to the correct booking/user/order	Done	The payment lookup is scoped to the caller. It used to match on the order id alone, so any authenticated user replaying somebody else's callback could drive the handler against a booking that was never theirs.
P-03	Signature verification server-side	Done	HMAC-SHA256 over <code>order_id payment_id</code> against the gateway secret, on the server. The handler refuses loudly if the secret is missing rather than treating every payment as a forgery.
P-04	Replay must not duplicate effects	Done	The lookup takes <code>FOR UPDATE</code> . A replay carries a <i>valid</i> signature, so signature checking alone cannot reject it; the lock makes the second arrival wait, and it then sees "already paid" and returns success with no booking update, no host earning, no notification and no ledger row. Before this, each replay created a duplicate earning — and host earnings feed the payout balance, so a replay was creditable money.
P-05	Failed payment must not confirm a booking	Done	Confirmation follows verification, never precedes it.
P-06	Pending state represented and reconciled	Done	Its own status, reconciled by the gateway webhook.
P-07	Refresh or repeated click must not duplicate	Done	Same lock, plus an order reused rather than recreated for the same booking.

8. Section 8 — Guest cancellation and refund

ID	Requirement	Status	Evidence
C-01	Policy displayed before confirmation	Done	<code>POST /user/cancel/quote</code> returns the exact refund for this booking under its own policy, and the page shows it before the guest confirms.
C-02	OTP confirmation required	Done	An email OTP; a request without one is answered <code>otpRequired: true</code> rather than proceeding.
C-03	Refund calculated server-side from the applicable policy	Done	From the policy snapshotted at booking , so a later change to the property's policy cannot alter what a guest was promised.
C-04	Cancelled booking cannot be modified	Done	The cancel query itself filters on the statuses from which cancellation is legal.
C-05	Guest sees read-only refund status	Done	<code>book_refund_status</code> on the booking.
C-06	Duplicate cancellation safely rejected	Done	The second attempt matches no row and is answered "Booking not found or can no longer be cancelled" — idempotent by construction rather than by a flag someone has to remember to check.

9. Section 9 — Guest stay, support and notifications

ID	Requirement	Status	Evidence
S-01	Stay information for an active/upcoming stay	Done	Location, amenities, rules and host contact are exposed to a guest with a booking — and, as of this review, no longer to everyone. See section 17.
S-02	Emergency information available	Done	Carried on the stay screen with the host and support contacts.
S-03	Guest can create a ticket with a unique Ticket ID	Built this review	The guest Support page offered FAQ links, a chat widget and an email address. There was no ticket and no reference number — a guest who wrote in could not tell whether anyone had read it. <code>POST /user/support/tickets/create</code> now returns a reference such as AJ-000123 .
S-04	Ticket status and updates remain visible	Built this review	The Support page lists the guest's tickets with status and an unread count, and opens the thread.
S-05	Closure follows the lifecycle; tickets are not silently deleted	Built this review	OPEN / PENDING / RESOLVED / CLOSED, and nothing on the guest side can delete a ticket or a message. A reply into a closed ticket is refused with the reference to quote, rather than accepted into silence.
S-06	Notifications correspond to real backend events	Done	Every notification is written by the flow it describes. A support reply now sends a guest to <code>/account/support</code> — it was hardcoded to the host portal, which a guest has no account for.
S-07	Messaging loads real conversations and enforces participation	Done	Scoped to the two parties.

10. Section 10 — Host registration, KYC and activation

ID	Requirement	Status	Evidence
H-01	Registration creates the correct host account	Done	
H-02	Identity/session verification end to end	Done	
H-03	KYC submitted securely and tied to the host	Done	Through the identity provider, keyed to the host record.
H-04	Pending/approved/rejected status accurate	Done	Gated on whether the verification is <i>current</i> , not on a session status an abandoned attempt leaves behind.
H-05	Inactive host cannot perform restricted actions	Done	Publishing a listing and requesting a payout both refuse an unverified host, with the reason.
H-06	KYC documents not reachable by guessed URLs	Done	Stored as Cloudinary <i>authenticated</i> assets, served only through short-lived signed URLs.

ID	Requirement	Status	Evidence
H-07	Host profile persists and displays	Done	

11. Section 11 — Host property management

ID	Requirement	Status	Evidence
H-08	Complete creation flow	Done	The five-step wizard, which is the only property form on web and app.
H-09	Host reaches only their own properties	Done	Every read and write filters on <code>property_host_id</code> from the token.
H-10	Data complete and validated	Done	Schema-driven, with a readiness score that has to reach 70 before submission.
H-11	Nightly/monthly pricing saved and used	Done	By booking and by negotiation.
H-12	Upload validation, limits and private document rules	Done	Type and size limits; identity and ownership documents are authenticated assets, never public.
H-13	Only eligible/verified listings become bookable	Done	Admin approval is the only thing that sets a listing live; submitting takes it <i>off</i> the site.
H-14	Unverified host cannot bypass restrictions	Done	A listing published for an unverified host is held inactive until they verify.
H-15	Changing a property id cannot expose another host's property	Done	Answered "Draft not found" — the same answer a missing id gets, so the error cannot be used to enumerate.

12. Section 12 — Host calendar and availability

ID	Requirement	Status	Evidence
H-16	Calendar only for owned properties	Done	Scoped by <code>property_host_id</code> .
H-17	Block eligible dates	Done	
H-18	Unblock where allowed	Done	
H-19	Existing bookings not overwritten by manual blocking	Done	A block that hits an occupying booking is refused; the occupancy rule is shared with the booking side so the two cannot disagree.

ID	Requirement	Status	Evidence
H-20	Guest availability reflects host changes	Done	Guest availability reads the same rules, not a cached copy.
H-21	Dates do not shift with browser timezone	Done	Dates are day-stamps in IST, computed by adding the IST offset and reading UTC fields — the same helper on the server, the website and the app, so the three cannot drift.

13. Section 13 — Host bookings

ID	Requirement	Status	Evidence
H-22	Only host-owned bookings shown	Done	
H-23	Guest/property/date/amount/status correct	Done	Money and dates are formatted on the host side too — they used to print raw API strings the guest side already formatted.
H-24	Accurate booking state	Done	One shared status vocabulary across the host, admin and guest screens, after a pass that found three screens giving one booking three different labels.
H-25	Only permitted guest information exposed	Done, and the mirror fixed	The host sees what they need to host. The reverse leak — the <i>host's</i> own phone and email on an unauthenticated endpoint — was found and fixed this review.
H-26	Host view updates on guest cancellation/refund	Done	

14. Section 14 — Host negotiation

ID	Requirement	Status	Evidence
H-27	Offers only for owned properties	Done	
H-28	Enough to respond, without cross-host data	Done	
H-29	Accept produces the correct final price and state	Done	
H-30	Counter produces a valid state and reaches the guest	Done	
H-31	Decline closes the offer	Done	
H-32	Cannot act on an expired offer	Done	
H-33	Concurrent responses cannot conflict	Done	The same locking read the guest side uses.

15. Section 15 — Host earnings, commission and payout

ID	Requirement	Status	Evidence
H-34	Earnings reflect eligible bookings and the approved commission	Done	Earnings are written by the payment flow, once, and the commission is one derivation rather than a figure each screen computes.
H-35	Available balance calculated server-side	Done — now one calculation	Received earnings minus everything already requested and not refused.
H-36	Host cannot request a payout for another host	Done	The host id comes from the token.
H-37	Payout restricted until KYC approval	Done	Money leaving the platform to a bank account nobody has identified is what verification is for.
H-38	Request above the available balance rejected	Fixed this review	There was a comment saying the amount should be checked against the host's account, and directly under it, the insert. Nothing checked anything — a host with fifty rupees of earnings could request fifty lakh, and because pending requests are subtracted from the balance a host is shown, one bogus request also drove their own displayed earnings negative.
H-39	Repeated request cannot duplicate	Fixed this review	A second pending request is refused, naming the one already waiting.
H-40	Pending/processing/completed/failed accurate	Done	Driven by the disbursement webhook.
H-41	Commission/earning calculation visible to the host	Done	Shown per booking and in the statement.

16. Section 16 — Cross-cutting web security

ID	Area	Result
SEC-01	Horizontal privilege escalation	Blocked. Guest bookings scope on the user id, host properties and bookings on <code>property_host_id</code> , payouts on the host id, tickets on both the user id and the ticket's own role. A wrong id is answered as <i>not found</i> , never as <i>forbidden</i> , so an error cannot be used to enumerate what exists.
SEC-02	Role escalation	Blocked. Admin routes take an admin token; a host or guest token is not one. Status columns are never read from a request body.
SEC-03	Price tampering	Blocked. Server recomputation on booking; negotiated prices are server-issued coupons; payment amounts are verified against the gateway signature.
SEC-04	Token/session security	Done. Real server-side revocation on logout, covered by tests.

ID	Area	Result
SEC-05	Private uploads	Done, and one leak closed. KYC, identity and ownership documents are authenticated assets behind signed URLs; invoices are no longer public static files. The host's own phone number and email were being returned by the unauthenticated property endpoint — every host's contact details could be harvested by walking property ids. Fixed.
SEC-06	Sensitive logs	Done. Request bodies are redacted by key — password, otp, pin, token, authorization, jwt, secret, apikey, signature and the KYC and bank fields — matched case- and separator-insensitively, so <code>cred_user_password</code> is caught as readily as <code>password</code> . A plaintext OTP that had been reaching a log line was removed on 5 September.
SEC-07	Rate limits	Fixed this review. The credential limiter — login, signup, OTP verification, password change — was running at 50 per 15 minutes , ten times its own documented value, because it had been raised "for demonstration" and never put back. The live API answered <code>ratelimit-limit: 50</code> ; it now answers 5 . Booking creation moved to its own budget of 30, because trying a few date combinations is not a credential guess.
SEC-08	CORS/CSRF	Done. An origin allowlist, not <code>true</code> , with a deliberate exception for requests carrying no Origin header at all (the app, webhooks). Authentication is a Bearer token rather than a cookie, so there is no ambient credential for a cross-site request to ride.
SEC-09	TLS	Done, and strengthened. Both hosts are HTTPS-only and http redirects to https. The API returned no HSTS header at all ; it now sends <code>max-age=31536000; includeSubDomains</code> , verified live.
SEC-10	Error leakage	Done, and tightened. A classifier replaces anything that smells of infrastructure — SQL text, unknown-column, duplicate-entry, connection errors — while letting through messages actually written for a person; when in doubt it replaces. A duplicate-key message on a phone number would otherwise confirm that number has an account. The API also advertised <code>X-Powered-By: Express</code> ; that is now off, along with nosniff, frame-deny and a referrer policy.

17. Section 17 — Mobile web

Measured with a real browser at **320 × 720**, the narrowest width the section names, on the live site.

Requirement	Result
Guest login/OTP usable at 320–412px	✓
Search filters, date picker, guest selector fit	✓ <code>/search</code> renders at exactly 320px with no overflow
Property gallery and details usable by touch	✓ and the photographs now carry ALT text, which they did not
Negotiation input visible with the keyboard open	✓
Booking/payment CTA not hidden behind sticky navigation	✓

Requirement	Result
Cancellation policy/OTP modal fits	✓
Guest booking details do not scroll horizontally	✓
Host dashboard, calendar and property forms at phone widths	Was failing — fixed. The host dashboard's action row measured 314px of buttons in 288px of space, so the page scrolled sideways. It wraps now.
Host negotiation controls accessible	✓
Host payout/KYC forms usable	✓
No horizontal scrolling at 320–360px	Two pages were failing — both fixed. <code>/host/dashboard</code> (330px of content in 320) and <code>/become-a-host</code> (378px in 320: a top bar that never got a phone treatment). Both measure exactly 320 now, verified live after deployment.
Orientation changes do not corrupt state	✓

Two wide tables — host bookings and host earnings — extend past the viewport **inside their own scrolling containers**, which is the correct answer rather than a defect: the page itself does not move.

18. Section 18 — API contract

ID	Requirement	Status
API-01	Bearer JWT lifecycle	Done , with server-side revocation.
API-02	Guest booking APIs integrated	Done — profile, list, detail, modify, cancel, refund quote, invoice, payment status.
API-03	Stay information	Done where exposed.
API-04	Support ticket create/update/get/list/close	Built this review for the guest; already present for the host. Both go into one admin queue.
API-05	Host APIs integrated	Done — profile, properties, detail, calendar, bookings, payouts, commission, KYC.
API-06	Notifications generated only from authorised backend flows	Done — every notification is written by the flow it describes; no client can raise one.
API-07	<code>AUTH_001</code> , <code>BOOKING_001</code> , <code>PAYMENT_001/002</code> , <code>REFUND_001</code> , <code>PROPERTY_001</code> , <code>SUPPORT_001</code> , <code>SERVER_500</code> handled safely	Needs your input — see below.
API-08	Retry policy; authentication and validation errors not blindly retried	Done — 5xx and network failures retry with backoff; 4xx does not.

API-07, plainly. The platform does not emit those symbolic codes. Every response is `{ success, message, data }` with an HTTP status, and errors are handled safely today — nothing leaks, and no client depends on a code. We can add a `code` field alongside the existing message, which is additive and safe for both the website and the app. **What we need from you is the code table itself** — which condition each of `AUTH_001` vs `AUTH_002`, and `PAYMENT_001` vs `PAYMENT_002`, refers to in your API Integration Specification. We are not willing to guess a mapping: a contract that disagrees with your specification is worse than one that has not been written yet. Send the table and this is a short piece of work.

19. What we changed during this review

Nine items, all deployed, all covered by tests.

1. An unauthenticated endpoint that sent email as Aajoo Homes

`POST /test/mail` took **no authentication** and sent an email from the Aajoo Homes account to whatever address was in the request body. It answered on production; we confirmed it was reachable without sending anything. Anyone who found it could send Aajoo-branded mail to anyone, at whatever rate the limiter allowed — a phishing template, and a fast route to the domain being blocklisted by the receiving providers. Removed, along with `/create/test`.

A test now fails the build if any *unauthenticated* test or debug route ever appears again. It deliberately permits the one guarded diagnostic that remains — an admin-only mail check, which exists because a failed send is otherwise invisible.

2. The credential rate limit was ten times its documented value

The limiter guarding login, signup, OTP verification and password change carried the comment *"5 requests per 15 minutes"* and the value **50**, with a note saying it had been raised for a demonstration. The live API confirmed it: `ratelimit-limit: 50`. It is 5 again, which also matches the 5-attempt limit on each OTP, so the per-code counter and the per-caller budget run out together. Booking creation was moved to its own budget of 30 rather than being dragged down with it.

3. Every host's phone number and email were public

`property_contact` and `property_email` are written from the host's own mobile and email at listing time, and the property endpoint is unauthenticated — so a scraper could walk `/properties/1`, `/properties/2` and collect every host's contact details. No guest-facing screen uses them; the property page offers a Message Host button, and the stay screen carries the contact once there is a booking. They are now hidden from anyone but the owner, beside the negotiation floor that was hidden for the same reason.

4. The API sent no security headers

No HSTS, no `nosniff`, no frame policy, and an `X-Powered-By: Express` banner. All four headers are set and the banner is off, verified live. Set by hand rather than by adding a security middleware, because that would also impose a content-security policy on a process that serves signed invoice files and a socket handshake.

5. WEB-P0-01 — the site was generating the URL it 404s

Described in section 2. The 404 was already correct; the link generation was not.

6 and 7. Two pages scrolled sideways on a 320px phone

The host dashboard and `/become-a-host`. Both measure exactly 320 now.

8. Guest support tickets

The whole feature, described in section 9. Same tables, same admin queue, a reference number the guest can quote, and a reply notification that sends them to the guest portal rather than to a host portal they have no account for. One migration, applied to the live database; the three existing tickets are hosts' and stay labelled as such.

9. A payout request was never checked against the balance

Described in section 15. The balance was already computed correctly — in the *read*. It is now one helper used by both the read and the create, because two answers to the same question about a host's money is a failure this platform has had before.

20. Section 19 — Final testing entry criteria

Criterion	Status
All P0 findings fixed	✓ six of six
All P1 findings on auth, authorization, negotiation, booking, payment, cancellation, KYC or payout fixed	✓
Developer regression completed	✓ 60/60 backend test files; the frontend production build is clean; the live checks in this document were re-run after deployment
Updated website and matching backend available	✓ frontend <code>b8f3818</code> , backend <code>1effc41</code> , both deployed
No development/test payment configuration in production	✓ the two test endpoints are gone; the payment gateway remains in test mode until you ask us to switch keys, which is a configuration change and not a deploy
Invalid property route fixed	✓
Guest booking journey end to end	✓
Host property/calendar/booking/negotiation/payout journey end to end	✓
Security negative tests completed	✓ section 16
Mobile web critical journeys pass	✓ section 17

21. Section 20 — Developer update

Required update	Developer response
Website build/version	b8f3818
Backend version/deployment	1effc41, deployed and verified live
P0 fixes completed	6 of 6
P1 fixes completed	All. Nine items were found untrue during this review and fixed; one feature (guest support tickets) was built.
Guest regression completed	Yes — search, booking, payment, cancellation, negotiation and account, by code review plus live checks on every unauthenticated surface
Host regression completed	Yes — listing, calendar, bookings, negotiation, earnings and payout
Payment regression completed	Yes — signature verification, replay, ownership, refresh, pending reconciliation
Negotiation regression completed	Yes — floor privacy, auto-accept, escalation, expiry, concurrency, the ledger
Security regression completed	Yes — section 16, including the four items fixed during it
Mobile web regression completed	Yes — measured at 320px on the live site before and after
Known remaining issues	One needs you: the API-07 error-code table. One needs your QA rather than ours: the new guest ticket flow was verified by build and by deployed bundle, but driving it end to end needs a signed-in guest session, and we do not type stored account passwords into forms. It is a ten-minute check and section 22 says how.

22. Release position

● **GO for final testing, with one flow for you to exercise.**

The document's blocking position rested on the six P0s and the security section. All six are addressed and the security items are measured rather than asserted — including four defects the review itself uncovered, one of which was live and exploitable by anyone who found the URL.

The ten-minute check we would like you to run first, because it is the one thing built rather than fixed here:

1. Sign in as a guest → **Account** → **Support**.
2. Raise a ticket. Confirm you are given a reference like `AJ-000123`.
3. In the admin panel → **Support**: the ticket appears in the same queue as host tickets, labelled **Guest**.
4. Reply from admin. The guest gets a notification that opens `/account/support`, not the host portal.

5. Reply back from the guest side; the ticket returns to the support queue.
6. Set it to **Closed** from admin. The guest sees it closed and is told to raise a new one quoting the old reference, rather than typing into a thread nobody will read.

Backend	<code>1effc41</code> — 60/60 test files
Frontend	<code>b8f3818</code> — production build clean
Migration	<code>20260906120000-guest-support-tickets</code> , applied to the live database
New tests	<code>tests/webHardening.test.js</code> (15 assertions), <code>tests/guestSupportTickets.test.js</code> (14)
Live verification	6 September 2026 — response codes, security headers, rate-limit headers, public API payloads and rendered widths at 320px